

Optimizing an Artificial Neural Network in PyTorch

Diagnosing Overfitting with Dropout, Batch Normalization, and L_2 Weight Decay

Abstract

A model can fit its training examples extremely well while making noticeably worse predictions on unseen examples. This difference is a generalization problem rather than a training-speed problem. These notes diagnose that problem for a Fashion-MNIST multilayer perceptron, compare seven possible responses, and develop three selected mechanisms from first principles: dropout, batch normalization, and L_2 regularization implemented through stochastic-gradient-descent weight decay. Every mechanism is connected to tensor shapes, a manual numerical example, PyTorch state transitions, executable code, and common failure modes. The final comparison reduces the reported training–held-out accuracy gap from approximately 9.3 to 4.9 percentage points while preserving held-out accuracy near 88%; careful interpretation distinguishes this smaller gap from an improvement in held-out accuracy.

Contents

1	Purpose and complete mental map	2
1.1	The practical problem	2
1.2	Optimization has two meanings	3
2	Seven responses to overfitting	3
3	Experimental contract: data, shapes, and baseline	5
3.1	Tensor shapes	5
3.2	Baseline architecture	5
3.3	Measure both partitions with one function	6
4	Dropout: stochastic hidden-unit masking	6
4.1	Purpose and vocabulary	6
4.2	Expanded numerical example	7
4.3	State, shapes, and failure modes	8
5	Batch normalization: feature-wise batch statistics	8
5.1	Purpose and exact operation	8
5.2	Expanded numerical example	9
5.3	Training and evaluation are different state transitions	9
6	L_2 regularization and SGD weight decay	10
6.1	The regularized objective	10
6.2	Expanded scalar update	11
6.3	The demonstrated optimizer call	11

7	Integrated architecture and parameter accounting	12
8	Correct training and evaluation state transitions	14
8.1	Training one epoch	14
8.2	Evaluation requires two independent controls	14
8.3	Complete run controller	15
9	How to interpret the combined result	16
9.1	What the numbers support	16
9.2	The “test” partition becomes validation data when reused	17
10	Verified execution and reproducibility checks	17
11	Failure modes and debugging checklist	18
12	Review questions	18

1 Purpose and complete mental map

1.1 The practical problem

Fashion-MNIST is a ten-class image-classification collection containing items such as shirts, trousers, coats, bags, and shoes. Each grayscale image has height 28 pixels and width 28 pixels. Flattening an image produces

$$28 \times 28 = 784$$

input values. A fully connected artificial neural network (ANN) maps those 784 values to ten *logits*, one unnormalized score for each class.

The working data table contains 60,000 images. An 80:20 split produces 48,000 training examples and 12,000 held-out examples. Training on the full table increased the previously reported held-out accuracy from roughly 83% to about 88.7%. A second measurement revealed the central defect: accuracy on training examples was approximately 98.0%. The model therefore learned the training collection much more closely than it generalized to held-out images.

Definition

Overfitting occurs when a fitted model captures training-specific patterns or noise strongly enough that performance on unseen data is meaningfully worse than performance on the training data. **Generalization** is the ability to perform well on examples that did not determine the fitted parameters.

Let A_{train} be training accuracy and A_{held} be held-out accuracy. A simple diagnostic is the *generalization gap*

$$G = A_{\text{train}} - A_{\text{held}}.$$

The symbol G measures a difference in accuracy. When accuracies are written as percentages, G is measured in *percentage points*, not percent. For the initial measurements,

$$G_{\text{before}} = 98.0\% - 88.7\% = 9.3 \text{ percentage points.}$$

A large positive value does not prove why a model fails, but it is strong evidence that simply fitting the training set more closely is not the main need.

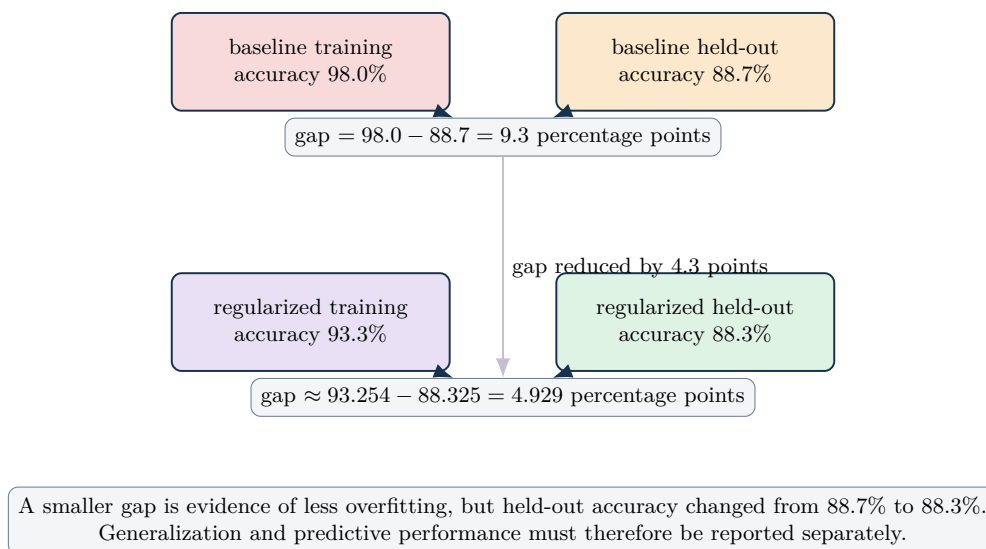


Figure 1: Training and held-out accuracy must be compared separately; the gap shrinks even though held-out accuracy changes slightly downward. [Technical reference](#).

Pitfall

Accuracy alone hides the error distribution and class imbalance. A complete study can add class-wise precision, recall, a confusion matrix, repeated splits, and uncertainty estimates. The accuracy gap remains useful here because both measurements use the same ten-class task and the same fitted parameters.

1.2 Optimization has two meanings

In mathematical machine learning, *optimization* usually means finding parameters that reduce an objective function. In practical model development, the word is also used more broadly for changing architecture, regularization, data, and training procedure to improve generalization. The selected changes affect both meanings:

- dropout changes the stochastic computation performed during training;
- batch normalization changes activation statistics and model state;
- L_2 regularization changes the objective or, equivalently for plain stochastic gradient descent, the parameter update.

Takeaway

The goal is not to maximize training accuracy. The goal is to learn parameters that retain predictive quality on unseen examples while avoiding unnecessary dependence on the training sample.

2 Seven responses to overfitting

Several mechanisms can address an excessive train–held-out gap. Their suitability depends on the task, data supply, architecture, and available compute.

1. **Collect or expose more representative data.** New examples constrain the model with more variation. The entire available 60,000-image table is already used, so no additional rows are introduced here.

2. **Reduce model capacity.** Fewer layers or neurons can reduce the set of functions the model can represent. The current ANN has only two hidden layers, with 128 and 64 units, so further simplification could cause underfitting.
3. **Add a parameter penalty.** L_1 or L_2 regularization discourages unnecessarily large parameters. The selected implementation uses L_2 weight decay.
4. **Use dropout.** Randomly masked hidden activations prevent units from relying on one fixed co-adaptation pattern in every training pass.
5. **Augment images.** Rotations, translations, crops, or flips can create task-preserving input variants. Augmentation is particularly natural for image models, including convolutional neural networks (CNNs), but it is deferred in this experiment. A flattened-input ANN can still consume augmented images; augmentation is not mathematically restricted to CNNs.
6. **Use batch normalization.** Mini-batch statistics stabilize activation scales and can introduce a mild regularizing effect through batch-to-batch noise.
7. **Stop training early.** A validation metric can determine when further epochs stop helping. Early stopping is deferred rather than combined with the three selected changes.

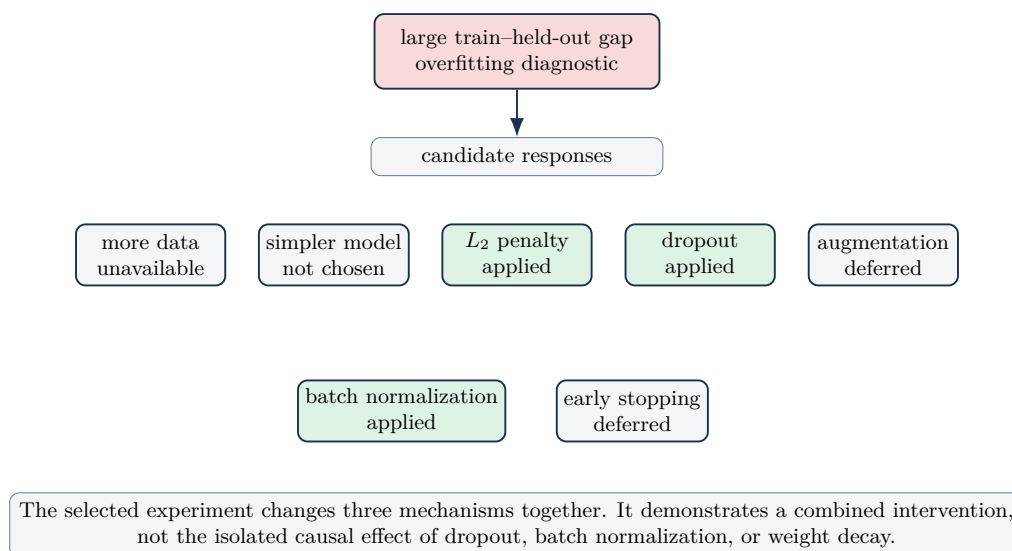


Figure 2: Seven possible responses are separated into applied, deferred, and unavailable choices for this experiment. [Technical reference](#).

Changing three mechanisms at once answers the practical question “Can this combination reduce the gap?” It does not answer “Which mechanism caused how much change?” That causal question requires an *ablation study*: hold every other setting fixed, introduce one mechanism at a time, and repeat across seeds.

Intuition

More data and data augmentation change the examples presented to the model. Capacity reduction and dropout change the effective function class. Batch normalization changes activation conditioning and state. L_2 regularization changes parameter preference. Early stopping changes how long optimization proceeds.

3 Experimental contract: data, shapes, and baseline

3.1 Tensor shapes

Let B denote mini-batch size. With $B = 32$, one feature batch and target batch have contracts

$$X_b \in \mathbb{R}^{32 \times 784}, \quad y_b \in \{0, 1, \dots, 9\}^{32}.$$

The matrix X_b stores one flattened image per row. The vector y_b stores one integer class index per example. Cross-entropy expects logits of shape $[32, 10]$ and targets of shape $[32]$ with integer data type `torch.int64`.

Because both split sizes are exact multiples of 32,

$$\frac{48,000}{32} = 1500 \quad \text{training batches per epoch}, \quad \frac{12,000}{32} = 375 \quad \text{held-out batches}.$$

With 100 epochs, stochastic gradient descent (SGD) performs

$$100 \times 1500 = 150,000$$

parameter updates.

```
import pandas as pd
import torch
from sklearn.model_selection import train_test_split
from torch.utils.data import DataLoader, TensorDataset

frame = pd.read_csv("fashion-mnist_train.csv")
X = frame.iloc[:, 1:].to_numpy(dtype="float32") / 255.0
y = frame.iloc[:, 0].to_numpy(dtype="int64")

X_train, X_held, y_train, y_held = train_test_split(
    X, y, test_size=0.2, random_state=42
)
train_data = TensorDataset(
    torch.from_numpy(X_train), torch.from_numpy(y_train)
)
held_data = TensorDataset(
    torch.from_numpy(X_held), torch.from_numpy(y_held)
)
train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
held_loader = DataLoader(held_data, batch_size=32, shuffle=False)
```

Listing 1: Load, split, scale, and batch the Fashion-MNIST table

Scaling maps integer pixel intensities from $[0, 255]$ to floating-point values in $[0, 1]$. The fixed split seed makes the partition reproducible, while `shuffle=True` changes training order from epoch to epoch.

3.2 Baseline architecture

The baseline ANN is a multilayer perceptron (MLP), meaning that it composes fully connected affine maps and nonlinear activation functions. A linear layer computes

$$H = XW^T + \mathbf{1}b^T.$$

Here $X \in \mathbb{R}^{B \times d_{\text{in}}}$ is an input batch, $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is the weight matrix, $b \in \mathbb{R}^{d_{\text{out}}}$ is the bias vector, and $H \in \mathbb{R}^{B \times d_{\text{out}}}$ is the output. The vector $\mathbf{1} \in \mathbb{R}^B$ repeats the same bias for every row.

```
import torch.nn as nn

class BaselineANN(nn.Module):
    def __init__(self, num_features=784):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(num_features, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 10),
        )

    def forward(self, features):
        return self.model(features)
```

Listing 2: Define the unregularized two-hidden-layer baseline

The shape sequence is

$$[B, 784] \rightarrow [B, 128] \rightarrow [B, 128] \rightarrow [B, 64] \rightarrow [B, 64] \rightarrow [B, 10].$$

ReLU, defined elementwise by $\text{ReLU}(u) = \max(0, u)$, changes values but not dimensions.

3.3 Measure both partitions with one function

A fair diagnostic uses exactly the same prediction rule for both loaders. `argmax(dim=1)` selects the index of the largest logit in each row.

```
def accuracy(model, loader, device):
    model.eval()
    correct = 0
    total = 0
    with torch.inference_mode():
        for features, targets in loader:
            features = features.to(device)
            targets = targets.to(device)
            predictions = model(features).argmax(dim=1)
            correct += (predictions == targets).sum().item()
            total += targets.numel()
    return correct / total

train_accuracy = accuracy(model, train_loader, device)
held_accuracy = accuracy(model, held_loader, device)
```

Listing 3: Measure accuracy without recording gradients

Shape and state check

The prediction tensor and target tensor both have shape $[B]$. A target shaped $[B, 1]$ is a column matrix rather than the required class-index vector and causes cross-entropy or comparison errors.

4 Dropout: stochastic hidden-unit masking

4.1 Purpose and vocabulary

Dropout reduces reliance on one fixed collection of hidden activations. It is applied to hidden representations during training, commonly after an activation function. The output layer is ordinarily left intact because its ten coordinates have direct class-score meaning.

Let p be the probability of dropping an activation and let

$$q = 1 - p$$

be its keep probability. For activation vector a and an independently sampled mask m with coordinates

$$m_i \sim \text{Bernoulli}(q),$$

PyTorch training-mode dropout computes

$$\tilde{a}_i = \frac{m_i a_i}{q}.$$

The symbol m_i is 1 with probability q and 0 with probability p . The division by q is called *inverted dropout scaling*.

The expected output equals the original activation:

$$\mathbb{E}[\tilde{a}_i] = \mathbb{E}\left[\frac{m_i a_i}{q}\right] = \frac{a_i}{q} \mathbb{E}[m_i] = \frac{a_i}{q} q = a_i.$$

Every symbol is now defined: a_i is one input activation, m_i is its random binary mask, p is drop probability, q is keep probability, and $\tilde{a}_i$ is the training-mode output.

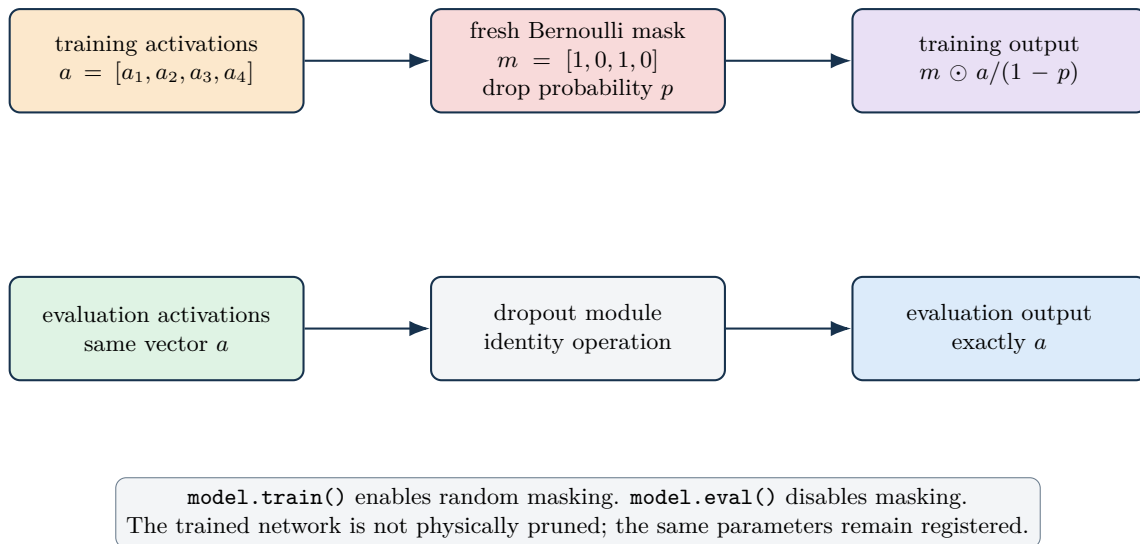


Figure 3: Dropout is stochastic in training mode and an identity map in evaluation mode. [Technical reference](#).

4.2 Expanded numerical example

Choose $p = 0.3$, so $q = 0.7$, and consider

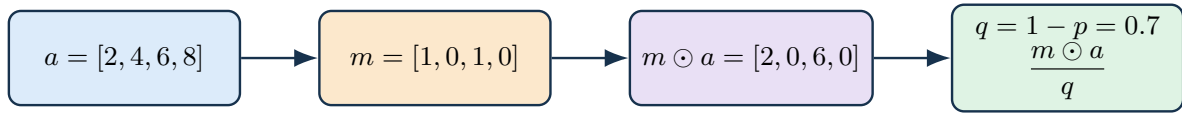
$$a = [2, 4, 6, 8], \quad m = [1, 0, 1, 0].$$

The elementwise product is

$$m \odot a = [1 \cdot 2, 0 \cdot 4, 1 \cdot 6, 0 \cdot 8] = [2, 0, 6, 0].$$

Training-mode scaling produces

$$\tilde{a} = \frac{[2, 0, 6, 0]}{0.7} \approx [2.857, 0, 8.571, 0].$$



One sampled training result is $[2.857, 0, 8.571, 0]$. For each coordinate, $\mathbb{E}[m_i a_i / q] = a_i$ because $\Pr(m_i = 1) = q$. Evaluation therefore needs no extra scaling.

Figure 4: A complete inverted-dropout calculation separates masking from keep-probability scaling. [Technical reference](#).

This one sample is not equal to a ; equality holds in expectation across masks. During evaluation, dropout returns a directly.

```
dropout = nn.Dropout(p=0.3)
x = torch.tensor([2.0, 4.0, 6.0, 8.0])

dropout.train()
torch.manual_seed(7)
training_sample = dropout(x)

dropout.eval()
evaluation_output = dropout(x)
assert torch.equal(evaluation_output, x)
```

Listing 4: Verify stochastic training behavior and deterministic evaluation behavior

4.3 State, shapes, and failure modes

`nn.Dropout(p=0.3)` preserves shape and data type. An input $[B, 128]$ becomes an output $[B, 128]$. The module has no trainable parameters. Its behavior depends on the parent module's state:

- `model.train()` enables random masks;
- `model.eval()` makes dropout an identity operation.

Pitfall

Forgetting `model.eval()` makes repeated evaluation calls stochastic. A very large p removes too much information and can cause underfitting. The demonstrated $p = 0.3$ is a hyperparameter, not a universal optimum.

5 Batch normalization: feature-wise batch statistics

5.1 Purpose and exact operation

Batch normalization (BatchNorm) controls the scale and location of hidden features. For a two-dimensional activation tensor $H \in \mathbb{R}^{B \times K}$, B is the mini-batch size and K is the number of hidden features. `nn.BatchNorm1d(K)` processes each feature column separately across the B rows.

For feature k , training-mode statistics are

$$\mu_{B,k} = \frac{1}{B} \sum_{i=1}^B h_{ik}, \quad \sigma_{B,k}^2 = \frac{1}{B} \sum_{i=1}^B (h_{ik} - \mu_{B,k})^2.$$

The symbol h_{ik} is feature k of example i , $\mu_{B,k}$ is the mini-batch mean, and $\sigma_{B,k}^2$ is the biased mini-batch variance used in the forward normalization. Standardization and the learnable affine transformation are

$$\hat{h}_{ik} = \frac{h_{ik} - \mu_{B,k}}{\sqrt{\sigma_{B,k}^2 + \varepsilon}}, \quad y_{ik} = \gamma_k \hat{h}_{ik} + \beta_k.$$

The small constant $\varepsilon > 0$ prevents division by zero. The trainable scale γ_k and shift β_k let the model choose a useful final distribution.

5.2 Expanded numerical example

Take one feature from two examples: $[1, 3]$. Its mean is

$$\mu_B = \frac{1+3}{2} = 2.$$

Its biased variance is

$$\sigma_B^2 = \frac{(1-2)^2 + (3-2)^2}{2} = \frac{1+1}{2} = 1.$$

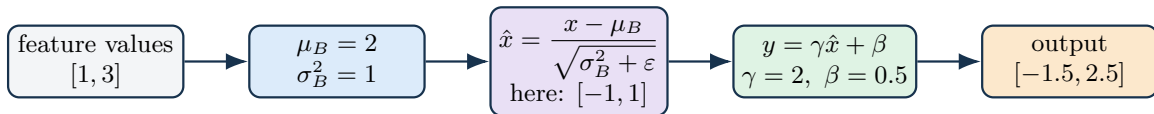
Ignoring the negligible ε only for this hand calculation,

$$\hat{h} = [(1-2)/1, (3-2)/1] = [-1, 1].$$

With $\gamma = 2$ and $\beta = 0.5$,

$$y = 2[-1, 1] + 0.5 = [-1.5, 2.5].$$

The result remains one value per example, so BatchNorm preserves the original $[B, K]$ shape.



Statistics are computed feature by feature across the mini-batch. The trainable scale γ and shift β allow the layer to recover a useful representation instead of forcing every feature to stay standardized.

Figure 5: BatchNorm standardizes each hidden feature across a mini-batch and then applies a trainable scale and shift. [Technical reference](#).

```
values = torch.tensor([1.0, 3.0])
mean = values.mean() # 2.0
variance = ((values - mean) ** 2).mean() # 1.0
normalized = (values - mean) / variance.sqrt()
gamma = torch.tensor(2.0)
beta = torch.tensor(0.5)
output = gamma * normalized + beta
assert torch.allclose(output, torch.tensor([-1.5, 2.5]))
```

Listing 5: Reproduce the two-example BatchNorm arithmetic explicitly

5.3 Training and evaluation are different state transitions

During training, BatchNorm uses current mini-batch statistics and updates non-trainable buffers called the *running mean* and *running variance*. During evaluation, it uses those accumulated running statistics. Evaluation does not bypass BatchNorm; it changes where the statistics come from.

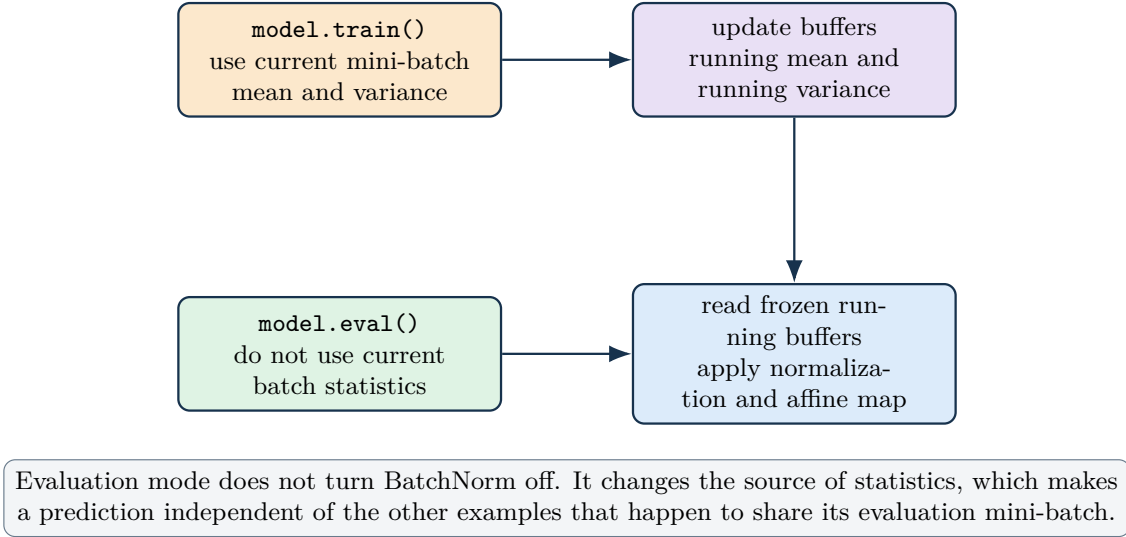


Figure 6: Training updates running statistics, whereas evaluation reads frozen running statistics rather than bypassing normalization. [Technical reference](#).

The historically popular explanation says BatchNorm reduces *internal covariate shift*, meaning changes in the distribution presented to later layers while earlier weights update. Modern evidence does not establish that reduction as the sole or necessary explanation. A safer interpretation is that BatchNorm improves optimization conditioning, stabilizes activation scales, permits useful learning-rate regimes, and introduces mini-batch-dependent noise that can have a regularizing effect.

Shape and state check

`BatchNorm1d(128)` receives $[B, 128]$, and `BatchNorm1d(64)` receives $[B, 64]$. The feature count in the constructor must equal the output width of the preceding linear layer.

Pitfall

Training-mode `BatchNorm1d` cannot estimate a feature variance from an input shaped $[1, K]$ and raises “Expected more than 1 value per channel.” If a final training batch can contain one example, choose a compatible batch size or use `drop_last=True`. The present 48,000/32 partition has no incomplete batch.

6 L2 regularization and SGD weight decay

6.1 The regularized objective

Let θ denote all model parameters and let w denote the subset of weight parameters to be penalized. A conventional L_2 -regularized objective is

$$J(\theta) = \mathcal{L}_{\text{data}}(\theta) + \frac{\lambda}{2} \sum_j w_j^2 = \mathcal{L}_{\text{data}}(\theta) + \frac{\lambda}{2} \|w\|_2^2.$$

The term $\mathcal{L}_{\text{data}}$ is cross-entropy on the current data, $\lambda \geq 0$ controls penalty strength, and $\|w\|_2^2$ is the sum of squared selected weights. The factor 1/2 simplifies differentiation:

$$\nabla_w J = \nabla_w \mathcal{L}_{\text{data}} + \lambda w.$$

Plain SGD with learning rate η then updates

$$w \leftarrow w - \eta (\nabla_w \mathcal{L}_{\text{data}} + \lambda w).$$

This update discourages large magnitudes because every selected weight receives a gradient component proportional to itself.

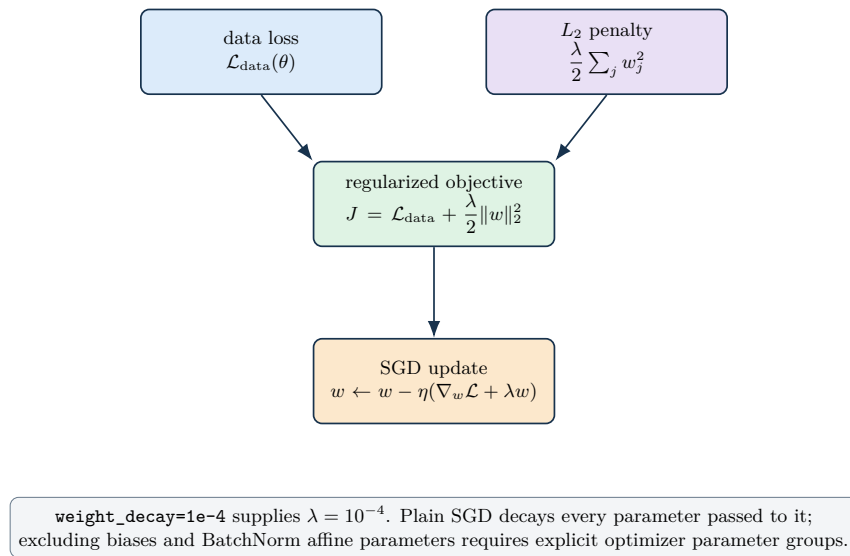


Figure 7: For plain SGD, L_2 regularization appears as a weight-decay term in the parameter update. [Technical reference](#).

6.2 Expanded scalar update

Suppose one parameter has value $w = 2$, data-loss gradient $g = 3$, learning rate $\eta = 0.1$, and decay coefficient $\lambda = 0.01$. The regularized gradient is

$$g + \lambda w = 3 + 0.01(2) = 3.02.$$

The updated parameter is

$$w_{\text{new}} = 2 - 0.1(3.02) = 2 - 0.302 = 1.698.$$

A direct PyTorch execution produces 1.6979999542, which agrees up to floating-point representation.

6.3 The demonstrated optimizer call

```

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(
    model.parameters(),
    lr=0.1,
    weight_decay=1e-4,
)
  
```

Listing 6: Enable the demonstrated SGD weight decay

The coefficient is

$$10^{-4} = 0.0001.$$

A larger value imposes a stronger shrinkage preference; a smaller value makes the penalty weaker. It must be tuned jointly with learning rate and architecture.

Pitfall

Passing `model.parameters()` applies PyTorch's SGD decay to every supplied trainable parameter, including linear biases and BatchNorm's γ and β . The optimizer cannot infer the conceptual label "weight only." Excluding one-dimensional bias and normalization parameters requires explicit groups.

```
decay = []
no_decay = []
for parameter in model.parameters():
    if parameter.ndim >= 2:
        decay.append(parameter)      # linear weight matrices
    else:
        no_decay.append(parameter)    # biases and BatchNorm gamma/beta

optimizer = torch.optim.SGD(
    [
        {"params": decay, "weight_decay": 1e-4},
        {"params": no_decay, "weight_decay": 0.0},
    ],
    lr=0.1,
)
```

Listing 7: Apply weight decay only to matrix-valued weights

For plain SGD, coupled weight decay and L_2 regularization have equivalent updates under the same convention. That equivalence must not be transferred blindly to adaptive methods. For example, AdamW deliberately decouples weight decay from the adaptive gradient update.

7 Integrated architecture and parameter accounting

The selected ordering for each hidden block is

Linear $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ Dropout.

BatchNorm acts on pre-activation linear outputs, ReLU introduces nonlinearity, and dropout masks the resulting hidden activations.

```
class RegularizedANN(nn.Module):
    def __init__(self, num_features=784):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(num_features, 128),
            nn.BatchNorm1d(128),
            nn.ReLU(),
            nn.Dropout(p=0.3),
            nn.Linear(128, 64),
            nn.BatchNorm1d(64),
            nn.ReLU(),
            nn.Dropout(p=0.3),
            nn.Linear(64, 10),
        )

    def forward(self, features):
        return self.model(features)
```

Listing 8: Define the complete regularized ANN

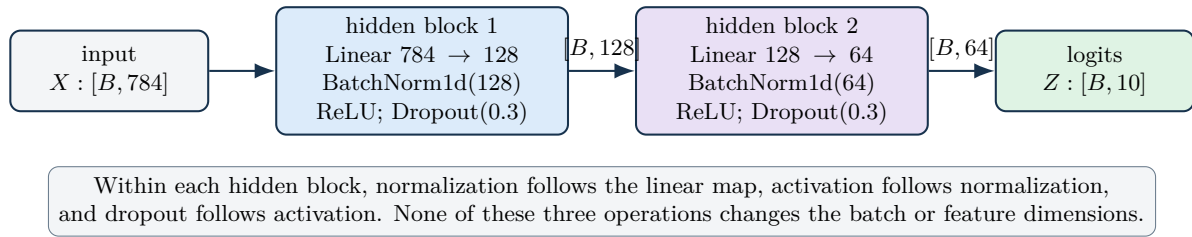


Figure 8: The complete architecture preserves batch size while changing feature width from 784 to 128, 64, and finally 10. [Technical reference](#).

For a batch of 32 examples, the detailed shape trace is

$$\begin{aligned}
 [32, 784] &\xrightarrow{\text{Linear}(784,128)} [32, 128] \xrightarrow{\text{BatchNorm1d}(128)} [32, 128] \\
 &\xrightarrow{\text{ReLU}} [32, 128] \xrightarrow{\text{Dropout}(0.3)} [32, 128] \\
 &\xrightarrow{\text{Linear}(128,64)} [32, 64] \xrightarrow{\text{BatchNorm1d}(64)} [32, 64] \\
 &\xrightarrow{\text{ReLU}} [32, 64] \xrightarrow{\text{Dropout}(0.3)} [32, 64] \xrightarrow{\text{Linear}(64,10)} [32, 10].
 \end{aligned}$$

Parameter counts expose what each module adds. The three linear layers contain

$$(784 \cdot 128 + 128) + (128 \cdot 64 + 64) + (64 \cdot 10 + 10) = 109,386$$

trainable scalars. The two BatchNorm layers add one γ and one β per feature:

$$2(128) + 2(64) = 384.$$

Dropout adds no parameters. The complete trainable count is therefore

$$109,386 + 384 = 109,770.$$

BatchNorm also owns running means, running variances, and batch counters. These are registered buffers rather than gradient-trained parameters.

```

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)
model = RegularizedANN().to(device)

features, targets = next(iter(train_loader))
features = features.to(device)
targets = targets.to(device)
logits = model(features)

assert features.shape == (32, 784)
assert targets.shape == (32,)
assert logits.shape == (32, 10)
assert sum(p.numel() for p in model.parameters()) == 109_770

```

Listing 9: Instantiate the model on one device and verify key contracts

8 Correct training and evaluation state transitions

8.1 Training one epoch

A training mini-batch performs six ordered actions: move tensors to the model's device, clear stale gradients, compute logits, compute cross-entropy, backpropagate, and update parameters. `model.train()` must be active so dropout samples masks and BatchNorm updates running statistics.

```
def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    loss_sum = 0.0
    example_count = 0

    for features, targets in loader:
        features = features.to(device)
        targets = targets.to(device)

        optimizer.zero_grad(set_to_none=True)
        logits = model(features)
        loss = criterion(logits, targets)
        loss.backward()
        optimizer.step()

        count = targets.numel()
        loss_sum += loss.item() * count
        example_count += count

    return loss_sum / example_count
```

Listing 10: Train for one complete epoch with state-dependent layers enabled

Multiplying each mini-batch mean loss by its example count before summing produces an example-weighted epoch mean. This remains correct even if a final batch is smaller.

8.2 Evaluation requires two independent controls

`model.eval()` changes Dropout and BatchNorm behavior. `torch.inference_mode()` disables autograd bookkeeping. Neither call substitutes for the other.

```
def evaluate(model, loader, device):
    model.eval()
    correct = 0
    total = 0

    with torch.inference_mode():
        for features, targets in loader:
            features = features.to(device)
            targets = targets.to(device)
            predictions = model(features).argmax(dim=1)
            correct += (predictions == targets).sum().item()
            total += targets.numel()

    return correct / total
```

Listing 11: Evaluate deterministically with Dropout and BatchNorm in inference state

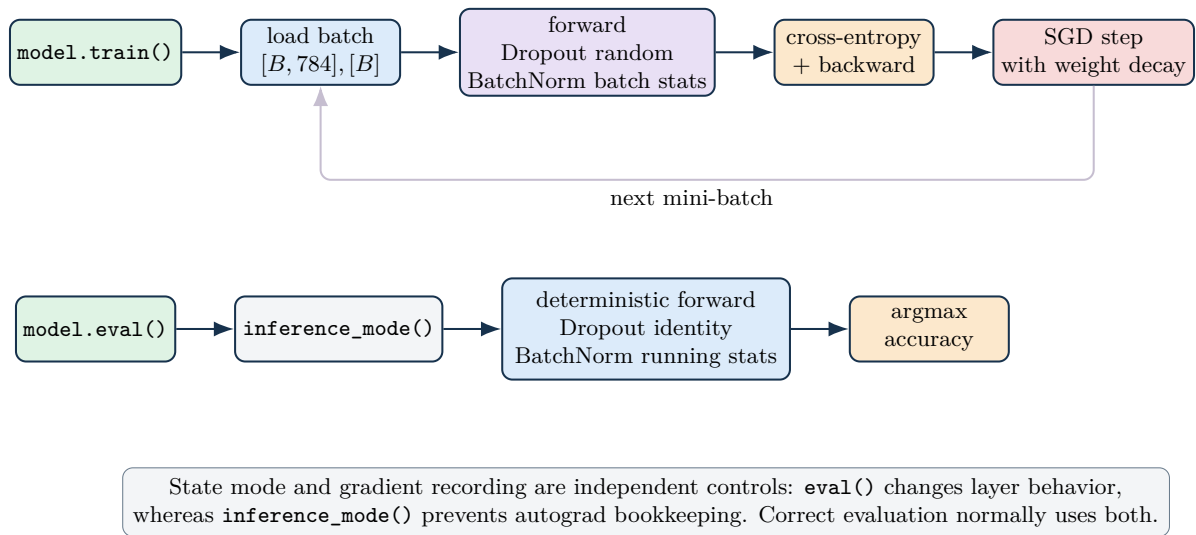


Figure 9: Training and evaluation use the same parameters but different layer state and gradient-recording rules. [Technical reference](#).

8.3 Complete run controller

```

epochs = 100
for epoch in range(epochs):
    mean_loss = train_one_epoch(
        model, train_loader, criterion, optimizer, device
    )
    print(f"epoch {epoch + 1:03d} | loss {mean_loss:.6f}")

train_accuracy = evaluate(model, train_loader, device)
held_accuracy = evaluate(model, held_loader, device)
gap_points = 100.0 * (train_accuracy - held_accuracy)

print(f"training accuracy: {100 * train_accuracy:.1f}%")
print(f"held-out accuracy: {100 * held_accuracy:.1f}%")
print(f"gap: {gap_points:.1f} percentage points")

```

Listing 12: Train for 100 epochs and then compare both partitions

The baseline and regularized evaluations produced exact displayed decimal accuracies. Converting each decimal to a percentage gives

$$\begin{aligned}
 A_{\text{train,before}} &= 0.9795625 = 97.95625\%, & A_{\text{held,before}} &= 0.8869166666666667 \approx 88.69167\%, \\
 A_{\text{train,after}} &= 0.9325416666666667 \approx 93.25417\%, & A_{\text{held,after}} &= 0.88325 = 88.325\%.
 \end{aligned}$$

The corresponding correct-prediction counts are 47,019/48,000 and 10,643/12,000 before the combined changes, then 44,762/48,000 and 10,599/12,000 afterward. Therefore,

$$\begin{aligned}
 G_{\text{before}} &= 100(0.9795625 - 0.8869166666666667) \approx 9.26458 \text{ points}, \\
 G_{\text{after}} &= 100(0.9325416666666667 - 0.88325) \approx 4.92917 \text{ points}, \\
 \Delta G &= G_{\text{before}} - G_{\text{after}} \approx 4.33542 \text{ points}.
 \end{aligned}$$

At one decimal place, the two gaps are 9.3 and 4.9 points, and the reduction is 4.3 points. The final six visible regularized-training losses were:

Epoch	Mean loss	Epoch	Mean loss
95	0.2598261367777983	98	0.26289867247144383
96	0.25824179737021524	99	0.2591799683657785
97	0.260292636650304	100	0.2601920379251242

Their small non-monotonic fluctuations are normal for mini-batch optimization; a single training-loss trace does not replace held-out evaluation.

Pitfall

Python function arguments and `nn.Sequential` modules must be comma-separated. A missing comma prevents the model or optimizer cell from running; fix syntax before interpreting any output. After evaluating, call `model.train()` again before resuming optimization.

9 How to interpret the combined result

9.1 What the numbers support

The train–held-out gap decreased substantially. That supports the statement that the fitted model is less overfit according to this diagnostic. Held-out accuracy changed from 88.69167% to 88.325%, a decrease of approximately 0.36667 percentage points. Training accuracy changed from 97.95625% to approximately 93.25417%. The run therefore does not demonstrate a held-out accuracy improvement. It demonstrates a large reduction in training accuracy, a small downward change in held-out accuracy, and a smaller gap.

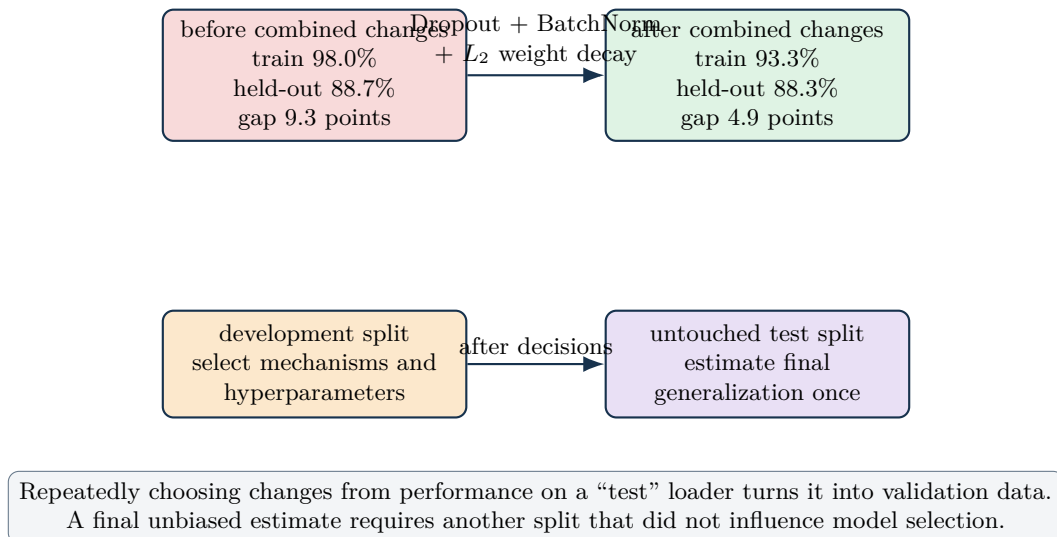


Figure 10: A smaller gap and a higher held-out score are distinct goals, and repeated development decisions require a separate final test split. [Technical reference](#).

The comparison changes dropout, BatchNorm, and weight decay simultaneously. It cannot identify the contribution of each. A rigorous ablation grid would include at least

1. baseline only;
2. baseline plus dropout;
3. baseline plus BatchNorm;
4. baseline plus weight decay;
5. selected pairs;
6. all three mechanisms.

Each configuration should use the same data partition, model widths, epochs, optimizer family, and several random seeds.

9.2 The “test” partition becomes validation data when reused

If held-out accuracy repeatedly determines architecture and hyperparameters, that partition participates in model selection. Its correct role is then *validation*, not final test. A stronger protocol uses three disjoint partitions:

Training set updates weights and BatchNorm running statistics.

Validation set selects dropout probability, decay coefficient, architecture, and stopping point.

Test set estimates final performance once, after all decisions are fixed.

This separation prevents optimistic bias caused by repeatedly adapting choices to one held-out set.

Takeaway

A reduced gap is valuable only when held-out performance remains useful. Report training accuracy, validation accuracy, gap, and final test accuracy rather than compressing them into one claim.

10 Verified execution and reproducibility checks

A complete executable validation run used the canonical 60,000-image Fashion-MNIST training collection. It used a deterministic 80:20 split, batch size 32, the complete regularized architecture, SGD with learning rate 0.1 and `weight_decay=1e-4`, and one full CPU training epoch. The execution verified:

- data shapes [60,000, 784], [48,000, 784], and [12,000, 784];
- exactly 1500 training batches and 375 held-out batches;
- logits shaped [32, 10] and 109,770 trainable parameters;
- two training-mode forwards differed because of dropout and batch statistics;
- two evaluation-mode forwards were exactly equal;
- BatchNorm running means changed in training mode;
- one complete CPU epoch processed all 48,000 training examples;
- the scalar SGD decay update produced 1.6979999542, matching the hand value 1.698;
- a one-example BatchNorm training batch and a $[B, 1]$ cross-entropy target raised the expected contract errors.

The one-epoch validation run reached training accuracy 0.8530833333 and held-out accuracy 0.8475. These values are smoke-test results rather than 100-epoch performance estimates. They establish that the architecture, shapes, state transitions, loss, and optimizer execute coherently. Reproducing a long-run metric additionally requires the same data ordering, random-number state, software stack, hardware, and full training schedule.

```
model.train()
train_a = model(features)
train_b = model(features)
assert not torch.equal(train_a, train_b)

model.eval()
with torch.inference_mode():
    eval_a = model(features)
    eval_b = model(features)
assert torch.equal(eval_a, eval_b)
assert eval_a.shape == (32, 10)
```

Listing 13: Make state and output invariants executable assertions

11 Failure modes and debugging checklist

- Dropout remains active during metrics.** Call `model.eval()` before validation or test inference.
- Wrong BatchNorm state.** Activate training mode while updating parameters. Activate evaluation mode while reporting metrics.
- Only autograd is disabled.** `inference_mode()` saves graph work but does not itself switch Dropout or BatchNorm state.
- Singleton BatchNorm batch.** Avoid a training input shaped $[1, K]$ for `BatchNorm1d`; use a compatible batch policy.
- Wrong BatchNorm width.** The constructor width must match the preceding linear output, such as 128 or 64.
- Wrong target contract.** Cross-entropy class indices require shape $[B]$ and integer type `torch.int64`.
- Excessive regularization.** A large p or λ can produce underfitting, lowering both training and held-out performance.
- Unexpected decay scope.** Default SGD decay reaches every parameter passed to the optimizer. Use parameter groups when biases or normalization parameters should be excluded.
- Confounded experiment.** Three simultaneous changes cannot establish individual causality; run ablations.
- Held-out leakage.** Do not repeatedly call a development partition “test” and then claim an unbiased final estimate.
- Missing comma.** Inspect adjacent modules and optimizer keyword arguments when Python reports a syntax error near a model-definition cell.
- Mode not restored.** After evaluation inside a training schedule, call `model.train()` before the next update.

12 Review questions

1. Why is $98.0\% - 88.7\% = 9.3$ measured in percentage points rather than percent?
2. For dropout probability $p = 0.3$, what is the keep probability and why are retained activations divided by it?
3. Starting from $a = [2, 4, 6, 8]$ and $m = [1, 0, 1, 0]$, reproduce the training-mode dropout output.
4. For values $[1, 3]$, compute the BatchNorm mean, biased variance, standardized values, and output when $\gamma = 2$ and $\beta = 0.5$.
5. What does `model.eval()` change in Dropout and BatchNorm? What does it not change about autograd?
6. Why does `weight_decay=1e-4` affect biases and BatchNorm affine parameters when all model parameters are passed to SGD?
7. Derive the scalar update from $w = 2$, $g = 3$, $\eta = 0.1$, and $\lambda = 0.01$.
8. Explain every shape in $[B, 784] \rightarrow [B, 128] \rightarrow [B, 64] \rightarrow [B, 10]$ and identify which operations preserve shape.
9. Why does a smaller generalization gap not automatically mean higher held-out accuracy?
10. Design an ablation study that isolates dropout, BatchNorm, and L_2 decay without reusing a final test set for selection.